RecordIndex 2.0: Estratégias Baseadas em LLMs Multimodais para Transcrição e Indexação de Registros Genealógicos Manuscritos

Alan Meniuk Gleizer[1]
Ivan Carlos Alcântara de Oliveira[1]

[1]Faculdade de Computação e Informática (FCI)
Universidade Presbiteriana Mackenzie São Paulo, SP – Brasil

alan.gleizer@mackenzista.com.br
ivan.oliveira@mackenzie.br

2026

Resumo

O RecordIndex é um sistema de transcrição e indexação automatizada de registros genealógicos manuscritos. Este trabalho compara três estratégias para essa tarefa: uma arquitetura de sete agentes apoiada em modelos de linguagem multimodais, lendo linha por linha ou a página inteira, e um modo generalista que transcreve, delimita os registros e extrai os campos em uma única chamada. Nove execuções sobre 132 registros de batismo do século XIX cruzam três modelos de características e desempenhos distintos com os três modos. Os resultados indicam que, nas configurações avaliadas, a decomposição em agentes não produziu ganho consistente de desempenho: com um modelo local de pequeno porte, é a granularidade da leitura que decide se o resultado é aproveitável; com o modelo proprietário de melhor desempenho, o generalista fica à frente das configurações multiagente. Na melhor configuração, 79,5% dos nomes são considerados utilizáveis pelo critério adotado, contra 51,5% na versão anterior, baseada em redes neurais convolucionais. O sistema é disponibilizado como aplicação web na qual as estratégias comparadas são selecionáveis pelo usuário.

Palavras-chave: reconhecimento de texto manuscrito; documentos históricos; arquiteturas de agentes; modelos de linguagem multimodais; extração de informação.

Abstract

RecordIndex is a system for the automated transcription and indexing of handwritten genealogical records. This work compares three strategies for this task: a seven-agent multi-agent architecture built on multimodal language models, reading either line by line or page by page, and a generalist mode that transcribes, delimits records and extracts fields in one call. Nine runs over 132 nineteenth-century baptism records cross three models of differing characteristics and performance with the three modes. The results indicate that, in the evaluated configurations, decomposition into agents did not produce a consistent performance gain: with a small local model, reading granularity decides whether the result is usable at all; with the best-performing proprietary model, the generalist comes out ahead of the multi-agent configurations. In the best configuration, 79.5% of names are considered usable under the adopted criterion, against 51.5% in the previous version, based on convolutional neural networks. The system is released as a web application, in which the compared strategies are user-selectable.

Keywords: handwritten text recognition; historical documents; agent architectures; multimodal language models; information extraction.

1 Introdução

Um desafio recorrente em Ciência da Computação é o tratamento de dados que, embora existam em formato digital, permanecem inacessíveis para pesquisa e análise automáticas. Bergman (2001) descreveu como *deep web* o conteúdo mantido em bases de dados que os mecanismos de busca não indexam e que, por isso, não pode ser explorado em escala. Acervos digitalizados de arquivos e bibliotecas estão nessa situação, e mesmo quando a imagem é acessível, não existe texto a ser indexado.

No âmbito da genealogia, documentos históricos manuscritos representam um caso especialmente difícil. Muitos registros civis e eclesiásticos continuam sendo digitalizados em bibliotecas e arquivos, mas sem transcrição. A ausência de texto pesquisável torna a consulta lenta e limita o potencial de exploração científica e social. No contexto da iniciação tecnológica, foi desenvolvido o *RecordIndex* (Gleizer; Oliveira, 2025), um *pipeline* que integrou redes neurais convolucionais (CNNs) para reconhecimento de texto manuscrito (HTR), empregando modelos de linguagem de larga escala (LLMs) para correção dos erros. Os resultados indicaram a viabilidade da abordagem, mas também revelaram limitações: desde a dificuldade de treinamento de modelos de HTR robustos até as barreiras relacionadas ao uso de LLMs enquanto ferramentas de correção textual.

Os experimentos deste trabalho utilizam o livro de registos de batismos da freguesia de Porto da Cruz, na ilha da Madeira, referente ao ano de 1863 (Biblioteca Pública e Arquivo Regional da Madeira, 1863). São 108 páginas manuscritas em português do século XIX, com caligrafia cursiva, tinta repassada entre páginas, anotações marginais e correções, contendo 132 assentos de batismo. Cada assento segue uma fórmula fixa que abre pela data e pelo local do ato, informa o nome do batizado, sua filiação e outros dados do ato. A coleção foi escolhida por reunir as condições necessárias à investigação: formato documental padronizado, imagens digitalizadas de acesso livre e um índice arquivístico que permite construir *ground truth* sem anotação manual.

A versão inicial do *RecordIndex* foi avaliada sobre esses mesmos 132 registros. Na transcrição inicial, 53 nomes foram classificados como corretos, 15 como reconhecíveis e 64 como erro. Após a etapa de correção com LLMs, o quadro passou para 47 corretos, 12

reconhecíveis e 73 erros (Gleizer; Oliveira, 2025). Esse resultado evidencia a necessidade de novas abordagens para melhorar a precisão do sistema, sobretudo na identificação de nomes próprios. Parte dessas limitações decorre da escassez de bases anotadas para o treinamento de modelos de HTR em português histórico. Poucos dados resultam em modelos frágeis, e o excesso de erros de transcrição reduz a eficácia das etapas posteriores, dificultando a segmentação de registros e aumentando as alucinações dos LLMs (Gleizer; Oliveira, 2025). Como cada língua e período histórico exige conjuntos anotados próprios, o esforço precisa ser repetido a cada novo contexto.

Diante desse quadro, o campo de HTR tem migrado para modelos de linguagem multimodais na transcrição a partir de imagens, que reduzem a dependência de bases anotadas e substituem abordagens como as CNNs utilizadas no *RecordIndex* (Humphries *et al.*, 2025; Kim *et al.*, 2025). Paralelamente, a pesquisa em agentes baseados em LLMs apresenta a decomposição de tarefas complexas em agentes especializados (Russell; Norvig, 2021) como alternativa modular a *pipelines* monolíticos (Xi *et al.*, 2023; Dorri *et al.*, 2018).

A partir dessas duas tendências, o *RecordIndex* foi redesenhado como uma arquitetura multiagente de agentes especializados. A evolução da capacidade dos modelos, porém, tornou necessário verificar empiricamente o benefício dessa decomposição. Para isso, foi implementado um modo generalista, com menor grau de decomposição, como configuração alternativa à arquitetura de sete agentes. O problema de pesquisa que orienta o trabalho é, portanto, *em que medida arquiteturas baseadas em LLMs multimodais podem reduzir a complexidade e ampliar a utilidade prática de pipelines tradicionais de HTR baseados em CNN na transcrição e extração de informações de registros genealógicos manuscritos.*

1.1 Objetivo Geral

Dessa forma, o objetivo geral deste Trabalho de Conclusão de Curso é avaliar comparativamente estratégias de transcrição e extração de informação de registros genealógicos manuscritos, investigando diferenças associadas ao grau de decomposição e os efeitos da granularidade de leitura em arquiteturas baseadas em LLMs multimodais, comparando-as à versão anterior baseada em CNNs e consolidando o *RecordIndex* como aplicação *web* acessível a pesquisadores sem conhecimento técnico em computação.

1.2 Objetivos Específicos

Para alcançar esse objetivo geral, foram definidos os seguintes objetivos específicos:

1. Avaliar LLMs multimodais como alternativa a CNNs na etapa de HTR, comparando modelos locais e comerciais, e verificar se a transcrição obtida dispensa os dados anotados para treinamento que limitavam a versão anterior.

2. Projetar e implementar as configurações comparadas, a arquitetura multiagente descrita pela especificação PEAS de Russell e Norvig (2021) e o modo generalista de menor grau de decomposição, com extração de campos via *structured output*, dispensando bibliotecas NER especializadas.

3. Definir um protocolo de avaliação quantitativa com métricas orientadas a campos, evitando comparações posicionais que se tornam inválidas após erros de segmentação, e executar as configurações sobre registros reais, investigando as diferenças associadas ao grau de decomposição e os efeitos da granularidade de leitura.

4. Consolidar o sistema como aplicação containerizada, com persistência em banco de dados e interface *web*, capaz de exportar os resultados e metadados.

A Seção 2 reúne os conceitos necessários para acompanhar as decisões de projeto, do reconhecimento de manuscrito às métricas empregadas na avaliação. A Seção 3 descreve o desenho comparativo, as configurações medidas, o protocolo de avaliação e o sistema construído. A Seção 4 apresenta as execuções, discute os achados e delimita seu alcance. A Seção 5 sintetiza as conclusões do trabalho.

2 Referencial Teórico

Esta seção apresenta os conceitos que fundamentam o trabalho: HTR e suas limitações, avanços recentes em LLMs multimodais, a padronização documental dos registros, extração de entidades, métricas de avaliação, agentes inteligentes e sistemas existentes.

2.1 Reconhecimento de Texto Manuscrito e Abordagens Baseadas em CNNs

O OCR (*optical character recognition*) foi projetado para textos impressos, em que os caracteres apresentam formas regulares, e mostrou-se inadequado para texto manuscrito, dada a alta variabilidade caligráfica e o ruído (Inunganbi, 2024; AlKendi *et al.*, 2024). A evolução foi o HTR, que trabalha tradicionalmente com sequências de caracteres, em nível de palavra, linha ou parágrafo (Garrido-Munoz *et al.*, 2025), e no qual redes neurais convolucionais (CNNs) se consolidaram como uma das principais abordagens (Inunganbi, 2024), por extraírem características robustas mesmo em contextos ruidosos.

A biblioteca *PyLaia* tornou-se referência para documentos históricos e utiliza CNNs como parte estrutural de sua arquitetura (Puigcerver, 2017; Tarride; Schneider *et al.*, 2024); sua combinação com a *doc-UFCN*, que também aplica CNNs para análise de *layout* e segmentação de linhas (Boillet *et al.*, 2021), constitui parte central de *workflows* recentes (Tarride; Maarand *et al.*, 2023) e sustentou a versão inicial do *RecordIndex*. O desempenho desses modelos depende, contudo, da disponibilidade de bases anotadas, e a escassez de texto manuscrito cursivo em português do século XIX foi o entrave que limitou a precisão daquele protótipo e comprometeu as etapas seguintes do *pipeline*.

2.2 Avanços em HTR a partir de LLMs Multimodais

LLMs foram inicialmente empregados como ferramentas de pós-correção de transcrições (Boros *et al.*, 2024; Kanerva *et al.*, 2025); os multimodais passaram então a ser investigados como substitutos diretos do reconhecedor. Humphries *et al.* (2025) e Kim *et al.* (2025) mediram taxas de erro inferiores às de sistemas especializados. A evidência favorável concentra-se em documentos em inglês: Crosilla *et al.* (2025) encontraram desempenho consistentemente pior nas demais línguas. Os cuidados apontados pela literatura recaem principalmente sobre a correção textual: Boros *et al.* (2024) indicam que LLMs não melhoram as transcrições e podem piorá-las, e Levchenko (2025) observou degradação após a correção em doze modelos multimodais. Kalra *et al.* (2024), por sua vez, obtiveram sucesso com correção por LLM, com aumento médio de 10% na acurácia de palavras reconhecidas.

Um aspecto pouco explorado nessa literatura é a unidade de imagem submetida ao modelo. Em geral, HTR opera em nível de linha, herança das arquiteturas recorrentes e convolucionais que exigem entradas de altura fixa (Tarride; Maarand *et al.*, 2023), ao passo que modelos multimodais aceitam a página inteira e dispõem, nesse caso, do contexto

gráfico e linguístico ao redor de cada trecho. Isom (2025) associa parte de seus ganhos justamente à ampliação do contexto fornecido ao modelo. Essa distinção, aqui denominada granularidade de leitura, é tratada neste trabalho como variável experimental.

2.3 Padronização Documental dos Registros Vitais

Diferente dos estudos citados, a proposta do *RecordIndex* lida com documentos de formato padronizado, e o trabalho parte da hipótese de que essa regularidade limita alucinações e serve de base para correções. Registros de batismo, casamento e óbito costumam ser lavrados segundo fórmulas fixas, repetidas de um assento para o outro, em que variam apenas os dados de cada evento. Nos assentos da coleção utilizada, o texto começa pela data e pelo local do evento, segue com o nome do registrado, a filiação e a profissão dos pais, e termina com testemunhas e autoridades. No *RecordIndex*, essa padronização facilita a detecção do início e do fim de cada registro e fornece um quadro de expectativa que restringe a atuação dos LLMs na correção.

2.4 NER e Extração Estruturada de Informações

O reconhecimento de entidades nomeadas (NER, do inglês *named entity recognition*) identifica automaticamente nomes, datas, localidades e outros elementos em texto. Em documentos históricos, o principal obstáculo é a escassez de dados anotados: o *survey* de Ehrmann *et al.* (2023) descreve os corpora disponíveis como poucos e dispersos entre línguas e períodos, de modo que cada nova coleção exige anotação própria. Abordagens supervisionadas reduziram esse custo. Abadie *et al.* (2022) compararam o *pipeline* de NER da biblioteca *spaCy*, baseado em redes convolucionais, com modelos baseados em *Transformers* em diretórios comerciais franceses do século XIX, e o melhor deles atingiu F1 de 92% com apenas 49 exemplos de treinamento. Ainda assim, alguma anotação continua necessária. Mais recentemente, LLMs mostraram-se capazes de realizar NER com boa precisão apenas via *prompting*, sem retreinamento (Hiltmann *et al.*, 2025), e este trabalho adota essa alternativa justamente por dispensar a anotação.

No *RecordIndex* 2.0 essa abordagem foi adaptada: em vez de marcar entidades no texto, o LLM preenche diretamente uma estrutura de dados pré-definida com os campos da coleção, realizando a extração estruturada em uma única etapa. Por *structured output* entende-se a capacidade de instruir um LLM a produzir saída em conformidade com um esquema, tipicamente declarado em JSON Schema. A vantagem prática sobre o NER supervisionado é a dispensa de dados anotados para treinamento, e o mesmo mecanismo funciona para qualquer coleção, bastando redefinir os campos.

2.5 Métricas de Avaliação

A avaliação de sistemas de OCR e HTR é tradicionalmente feita por métricas de distância de edição, sobretudo *Character Error Rate* (CER) e *Word Error Rate* (WER) (Garrido-Munoz *et al.*, 2025), que contabilizam as operações necessárias para transformar a transcrição automática no *ground truth*. Elas dependem de *ground truth* em larga escala, e não bastam sozinhas para caracterizar a qualidade de uma transcrição (Neudecker *et al.*, 2021). Além disso, CER e WER tratam todos os caracteres ou palavras de forma equivalente, e em genealogia um erro em nome próprio compromete o registro muito mais do que um erro em palavra comum. Por isso este trabalho adota métricas orientadas a entidades, construídas sobre *precision*, *recall* e *F1-score* conforme definidos por Manning *et al.* (2008), e mede a similaridade entre nomes por Jaro-Winkler, detalhados na Seção 3.4.

2.6 Agentes Inteligentes e Sistemas Multiagente

Um agente inteligente percebe seu ambiente por meio de sensores e age sobre ele por meio de atuadores, e sua tarefa é especificada pelo modelo PEAS (*Performance, Environment, Actuators, Sensors*), que explicita os critérios de sucesso, o ambiente de operação e os mecanismos de ação e de percepção (Russell; Norvig, 2021). Na arquitetura proposta, os agentes são caracterizados, conforme os mesmos autores, como reativos simples, reativos baseados em modelo ou baseados em objetivos, de acordo com o uso de estado e de metas na seleção de suas ações. Um sistema multiagente (MAS, do inglês *multi-agent system*) reúne múltiplos agentes interagindo em um ambiente compartilhado (Dorri *et al.*, 2018). No *RecordIndex* os agentes cooperam no processamento, cada um responsável por uma etapa especializada. O uso de LLMs como motor de decisão em agentes é tendência recente, mapeada por Xi *et al.* (2023).

2.7 Trabalhos Relacionados e Sistemas Existentes

Entre as plataformas consolidadas que buscam resolver problemas semelhantes destacam-se três. O *Transkribus* (Muehlberger *et al.*, 2019) concentra-se na transcrição automática, combinando modelos de HTR treinados com ferramentas de segmentação e correção; apresenta alta precisão e ampla base de usuários, mas o módulo de reconhecimento não é aberto, e o serviço opera em modelo *freemium*, o que limita sua reprodutibilidade e acesso. O *Arkindex*, da empresa *Teklia*, adota abordagem modular e aberta para processamento documental em larga escala (TEKLIA, 2023). O *eScriptorium* é uma plataforma de código aberto que integra o sistema *Kraken* (Kiessling *et al.*, 2019), voltada à pesquisa acadêmica e distribuída de forma inteiramente livre.

A Tabela 1 sintetiza as características que diferenciam essas plataformas da proposta (TEKLIA, 2024). A distinção mais relevante está na última linha: todas as plataformas operam executando um modelo de HTR treinado, próprio ou público, enquanto a proposta emprega modelos multimodais de uso geral, sem treinamento específico para a coleção.

Característica	**Arkindex**	**Transkribus**	**eScriptorium**	***RecordIndex* 1.0**	***RecordIndex* 2.0**
Extração de entidades	✓	–	–	–	✓
Código aberto	✓	–	✓	✓	✓
Hospedagem própria	✓	–	✓	✓	✓
Acesso via Web	✓	✓	✓	–	✓
Depende de modelo treinado	✓	✓	✓	✓	–

Tabela 1 – Comparação entre plataformas (adaptado de TEKLIA (2024))

3 Metodologia

A pesquisa é de natureza aplicada, pois busca solucionar um problema específico: a transcrição e indexação automatizada de documentos genealógicos manuscritos. O propósito tem caráter exploratório, voltado à experimentação e combinação de técnicas de modelos de linguagem de larga escala, reconhecimento de entidades nomeadas e arquiteturas de agentes, para avaliar sua integração no contexto proposto. A abordagem é quantitativa, com base em métricas objetivas (*precision*, *recall* e *F1-score*) que permitem mensurar e comparar o desempenho entre diferentes configurações. A pesquisa foi conduzida em ambiente laboratorial e classificada como experimental, dado o controle das variáveis envolvidas e a execução sistemática de testes comparativos.

As subseções seguintes percorrem, nessa ordem, a trajetória que levou ao desenho experimental, as configurações comparadas, a seleção de modelos, o protocolo de avaliação e o artefato de software resultante.

3.1 Trajetória da Pesquisa e Desenho Experimental

Na etapa inicial do projeto os modelos multimodais não transcreviam manuscrito cursivo de forma aproveitável, e restava aos LLMs o papel de corretor aplicado após a transcrição (Gleizer; Oliveira, 2025): as CNNs eram a única alternativa viável para o HTR, e a divisão em etapas rígidas decorria das ferramentas, não apenas de uma decisão de projeto. Em um segundo momento os modelos passaram a transcrever bem o bastante para substituir as CNNs, mas ainda falhavam em tarefas compostas numa única chamada, o que tornou natural decompor o processamento em agentes especializados. Mais recentemente eles passaram a resolver a tarefa composta inteira de uma vez, e então a pergunta sobre o benefício da decomposição tornou-se necessária e testável.

Por isso, a comparação entre o *RecordIndex* 1.0 e as configurações baseadas em LLMs é indireta: os dois sistemas foram avaliados em momentos diferentes e diferem em reconhecedor, em estratégia de segmentação e em critério de avaliação, de modo que a comparação não isola o componente de HTR. O que é efetivamente controlado é a comparação entre as configurações baseadas em LLMs, executadas sobre a mesma coleção, com o mesmo *ground truth*, o mesmo módulo de avaliação e o mesmo modelo de linguagem. Essa separação entre o que é comparação controlada e o que é comparação indireta é retomada na Seção 4.6.

3.2 Configurações Comparadas

O desenho experimental considera duas dimensões que costumam ser tratadas em conjunto na literatura. A primeira é o grau de decomposição, isto é, em quantos componentes especializados a tarefa é dividida, indo do *pipeline* de etapas fixas do *RecordIndex* 1.0 aos sete agentes da arquitetura multiagente e ao modo generalista. A segunda é a granularidade de leitura, isto é, se o modelo recebe uma linha isolada ou a página inteira a cada chamada. A Tabela 2 sintetiza cada configuração.

Configuração	Componentes	Leitura	Chamadas por página	Continuidade entre páginas	Treino
1.0 (CNN)	4 etapas fixas	linha	uma por linha, mais correção	texto acumulado	exige
Multiagente por linha	7 agentes	linha	uma por linha, mais A3–A6 por registro	texto acumulado pelo A0	dispensa
Multiagente por página	7 agentes	página	uma por página, mais A3–A6 por registro	texto acumulado pelo A0	dispensa
Generalista	3 agentes	página	uma por página, mais uma por fronteira	mesclagem decidida pelo AM	dispensa

Tabela 2 – Configurações, com grau de decomposição e granularidade de leitura.

Uma premissa é compartilhada por todas as configurações: o formato padronizado dos registros, com fórmulas fixas intercaladas por campos variáveis, favorece tanto a delimitação de registros, pelos inícios padronizados, quanto a extração, pela posição previsível dos campos (Seção 2.3). Todas as configurações exploram essa regularidade, e os resultados que se seguem valem para documentos com esse grau de padronização.

Comparar a leitura por linha com a leitura por página dentro da arquitetura multiagente isola a granularidade: os modelos são os mesmos, a coleção é a mesma, e só muda o que o modelo de transcrição recebe a cada chamada. Os dois modos operam com os mesmos sete agentes, de modo que nada além da granularidade varia entre eles. Já a comparação entre a arquitetura multiagente lendo por página e o modo generalista mantém

a granularidade fixa, mas não isola um único fator. Além do número de agentes, mudam a organização das operações, que passam a ocorrer em uma chamada única; a forma de delimitação dos registros, decidida sobre a imagem da página no generalista e sobre o texto já transcrito no multiagente; a correção estrutural do A4, ausente no generalista; e o tratamento da fronteira entre páginas, por acúmulo de texto no A0 ou por mesclagem decidida pelo AM. O que essa comparação contrasta, portanto, são dois arranjos completos, e não o grau de decomposição isolado.

3.2.1 *RecordIndex* 1.0: *pipeline* convolucional

A versão inicial do sistema organizava o processamento em quatro etapas independentes, cada uma sustentada por uma ferramenta própria (Gleizer; Oliveira, 2025). A análise de *layout* e a segmentação de linhas eram realizadas pela biblioteca *doc-UFCN* (Boillet *et al.*, 2021); o reconhecimento de cada linha ficava a cargo do *PyLaia* (Puigcerver, 2017), treinado sobre dados anotados; a detecção de início de registro comparava cada linha transcrita a uma linha de referência por similaridade de cossenos; e um LLM aplicava correção textual sobre o resultado, antes da exportação.

O *RecordIndex* 1.0 já dividia a tarefa em etapas, e de forma mais rígida do que as configurações comparadas: cada etapa tinha contrato explícito de entrada e saída e era treinada ou parametrizada separadamente. Na versão 2.0, as etapas passam a ser executadas por LLMs, e cada uma pode assumir uma parte maior da tarefa; no modo generalista, uma única chamada por página transcreve, delimita os registros e extrai os campos.

3.2.2 Arquitetura multiagente: agentes A0–A6

O *RecordIndex* 2.0 foi redesenhado como um sistema multiagente cooperativo, com agentes especializados operando sob coordenação de um orquestrador. A especificação PEAS (Russell; Norvig, 2021) foi utilizada para descrever as responsabilidades, entradas e saídas de cada agente. A Figura 1 sintetiza a arquitetura, separando o orquestrador, que mantém estado ao longo da coleção, dos agentes de execução, que não mantêm memória.

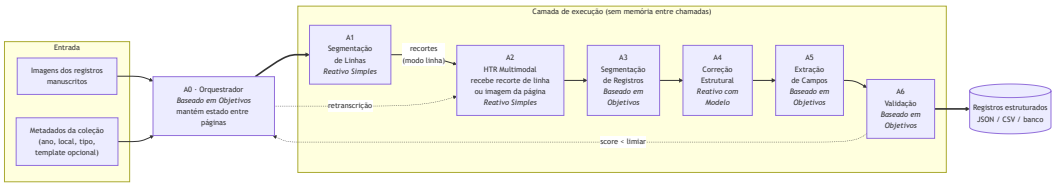


Figura 1 – Arquitetura multiagente do *RecordIndex* 2.0.

O A0 classifica o tipo de coleção (batismo, casamento ou óbito, por exemplo) e coordena a execução dos demais. Para cada página, o A1 segmenta as linhas de texto via *doc-UFCN* (Boillet *et al.*, 2021), o A2 transcreve o conteúdo com um LLM multimodal, recebendo a imagem de uma linha isolada ou da página inteira conforme o modo de leitura, e o A3 identifica as fronteiras entre registros a partir do texto transcrito. Fechado um registro, o A4 corrige seu texto com base em um *template* do assento fornecido pelo usuário, o A5 extrai os campos via *structured output* preenchendo um *schema* dinâmico definido pela configuração da coleção (Hiltmann *et al.*, 2025), e o A6 valida o resultado. A Tabela 3 apresenta a especificação PEAS de cada agente.

Tabela 3 – Especificação PEAS dos sete agentes.

Agente	***Performance***	***Environment***	***Actuators***	***Sensors***
A0 Orquestrador	Coleção íntegra com registros completos e validados	Coleção de imagens, metadados e saídas dos demais agentes	Classificação da coleção; invocação de agentes; reexecução	Metadados; nomes de arquivo; saídas de A1–A6
A1 Segm. linhas	Linhas com altura e razão de aspecto adequadas	Img. de página e estatísticas locais	*Crops* de linhas; *flag* `is_valid`	Imagem de página
A2 HTR	Texto fiel ao manuscrito	Img. de linha ou página	Texto plano transcrito	Imagem recebida
A3 Segm. registros	Fronteiras corretas entre registros	Transcrições e indício de início de registro	Índices de início de registro	Transcrições de A2
A4 Correção	Texto normalizado conforme *template*	Texto bruto e *template* com *placeholders*	Texto corrigido	Texto bruto; *template* da coleção
A5 Extração	Campos preenchidos conforme *schema*	Texto do registro e *schema* dinâmico	JSON com campos estruturados	Texto do registro e *schema*
A6 Validação	*Score* próximo de 1 para registros corretos	Campos extraídos e texto transcrito	*Score* de 0 a 1; veredito; lista de erros	Saída de A5; saída de A2

O modelo de dados é hierárquico, da coleção à página, à linha e ao registro, com cada linha pertencendo simultaneamente a uma página, por proveniência física, e a um registro, por agrupamento semântico. Essa estrutura permite que registros se estendam por múltiplas páginas e que uma mesma página contenha vários registros. O A0 gerencia essa continuidade acumulando o texto de um registro em aberto entre páginas sucessivas e extraindo os campos uma única vez, quando o fechamento do registro é confirmado.

O A0 monta, no começo da execução, uma configuração de coleção que os demais agentes consomem: um indício textual de início de registro, que orienta o A3; um *template* do assento, que é usado pelo A4; e a lista de campos, que define o *schema* do A5. O indício é derivado das primeiras palavras do *template* fornecido pelo usuário. Mudar de batismos para casamentos ou óbitos não exige, portanto, alterar a lógica interna de nenhum agente.

O A0 implementa também um *feedback loop*: registros com pontuação A6 abaixo de um limiar configurável são retranscritos com um modelo alternativo, o que permite combinar um modelo local como primário com um modelo maior como *fallback*. Nos experimentos comparativos esse mecanismo foi desativado, para que cada execução utilizasse um único modelo. A coordenação entre agentes foi implementada diretamente em Python, sem *frameworks* de orquestração, e a integração com os modelos foi feita com o *LangChain*.

3.2.3 Modo generalista

O modo generalista atribui a um único agente generalista (AG) a tarefa completa. Ele recebe a imagem da página inteira e devolve, em uma única chamada, os registros já delimitados e com os campos preenchidos, sinalizando quando um registro se encontra cortado pela borda inferior da página. Leitura, segmentação e extração, distribuídas entre A2, A3 e A5 na arquitetura multiagente, ocorrem aqui de forma conjunta. O agente generalista recebe as mesmas informações de coleção que o modo multiagente.

Um segundo componente, o agente de mesclagem (AM), trata exclusivamente da fronteira entre páginas. Como um registro pode ocupar mais de uma página e uma página pode conter mais de um registro, é necessário decidir se o primeiro registro de uma página é continuação do registro deixado em aberto na página anterior ou um registro novo. Essa decisão depende de interpretar o conteúdo, ela foi atribuída a um agente, e não a uma regra determinística. A Figura 2 apresenta a configuração, mantendo a mesma linguagem gráfica da Figura 1 para permitir a leitura comparativa. Com o orquestrador responsável pelo fluxo, essa configuração totaliza três agentes: orquestrador, AG e AM.

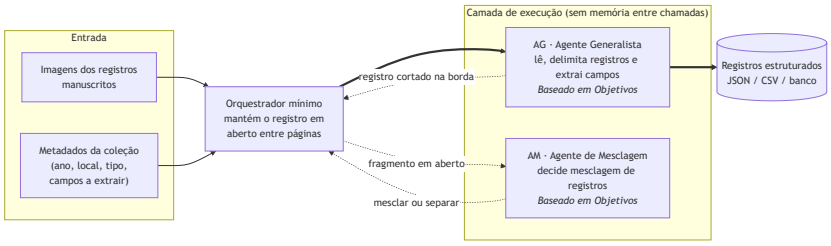


Figura 2 – Arquitetura do modo generalista.

3.3 Seleção de Modelos

A escolha de modelos seguiu critérios de capacidade multimodal, custo e viabilidade de execução local. Testes preliminares sobre uma amostra reduzida da coleção de batismos de Porto da Cruz (Biblioteca Pública e Arquivo Regional da Madeira, 1863) mostraram que modelos locais de pequeno porte comprometem toda a cadeia quando empregados no A2: o *gemma4:e4b* produziu F1 de segmentação de 0,13. A qualidade da transcrição, portanto, condiciona tudo o que vem depois.

Para segmentação e extração, que operam sobre texto já transcrito, modelos menores demonstraram desempenho adequado, o que abre a possibilidade de configurações de modelo misto, mais baratas. Elas ficaram fora do desenho comparativo, que mantém um único modelo por execução. Foram selecionados três modelos com características e desempenhos distintos, de modo a verificar se as conclusões arquiteturais dependem do modelo empregado: o *qwen3.5:9b*, executado localmente via Ollama, como modelo local de menor porte; o *qwen3.5:397b*, o maior modelo aberto do catálogo consultado; e o *Claude Sonnet 4.6*, modelo proprietário de melhor desempenho global entre os avaliados. A ordem de apresentação não representa uma escala controlada de capacidade. Os três foram executados sobre a coleção completa nos três modos. O *qwen3.5:397b* não coube na memória da GPU descrita na Seção 3.5 e foi acessado pelo serviço em nuvem do Ollama, de modo que é aberto quanto aos pesos, mas não foi executado localmente nos experimentos.

3.4 Protocolo de Avaliação

A avaliação reporta separadamente a segmentação de registros e a extração de campos, permitindo identificar em qual etapa se concentram as falhas. A necessidade vem de um problema prático: se o sistema divide um assento em dois, ou funde dois em um, tudo o que vem depois se desloca, e o registro N da saída deixa de corresponder ao registro N do *ground truth*: a comparação passa a ser entre pessoas diferentes. Por isso o casamento entre saída e referência é feito por similaridade de conteúdo, e não apenas por posição. Nos registros de batismo, esse casamento considera os campos nome, pai e mãe. O módulo de avaliação opera como serviço independente: recebe o JSON do *pipeline* e um CSV de referência com os campos nome, pai, mãe e data.

O casamento entre a saída do *pipeline* e o *ground truth* usa a similaridade de Jaro-Winkler, que pontua duas cadeias de 0 a 1 e favorece as que compartilham o início. Cohen *et al.* (2003) avaliaram diversas métricas de similaridade em tarefas de casamento de nomes e indicam as variantes de Jaro como apropriadas a cadeias curtas, como nomes próprios, com desempenho competitivo entre as métricas de distância de edição e custo computacional menor.

Na camada de segmentação de registros, *precision*, *recall* e F1 são calculados sobre registros casados contra o *ground truth*, e a razão entre o total produzido e o total de referência complementa a leitura: acima de 1,0 indica hipersegmentação, abaixo de 1,0 indica fusão.

Na camada de extração, as métricas são calculadas por campo (nome, pai, mãe, data) sobre os pares casados, com limiar de 0,85 JW para verdadeiro positivo, e adota-se a semântica de Makhoul *et al.* (2000): um campo preenchido com valor incorreto contabiliza FP e FN, pois houve erro de comissão ao produzir algo errado e de omissão ao deixar de produzir o valor correto, enquanto um campo ausente contabiliza apenas FN. A consequência prática é que preencher campos com valores errados, em vez de deixá-los em branco, não infla o *recall*.

Há ainda um viés próprio de *pipelines* encadeados. Quando a segmentação erra e um registro do *ground truth* fica sem par, ele desaparece do cálculo de extração, que passa a medir apenas os registros que o sistema conseguiu delimitar e, por isso, sobe. Para corrigir esse efeito, este trabalho define a taxa de recuperação efetiva por campo como a razão entre os verdadeiros positivos e o total de registros do *ground truth*, incluindo no denominador registros casados ou não. Ela é uma variante do *recall*, e o que a distingue é o denominador: o *recall* da camada de extração considera apenas os pares casados, e o da camada de segmentação mede a cobertura dos registros, não dos campos. Combinada com a *precision* medida sobre os pares casados, ela dá o $F1_{\text{ef}}$, indicador composto definido neste trabalho para integrar as duas camadas em um único valor:

$$R_{\text{ef}} = \frac{TP}{N_{\text{GT}}}, \quad F1_{\text{ef}} = \frac{2\,P\,R_{\text{ef}}}{P + R_{\text{ef}}} \tag{1}$$

onde N_{GT} é o total de registros do *ground truth* e P é a *precision* sobre os pares casados. Como R_{ef} tem por denominador o total do *ground truth*, e não os pares casados, o $F1_{\text{ef}}$ não é o F1 da camada de extração e não deve ser comparado diretamente a valores de F1 reportados na literatura.

3.5 Infraestrutura e Aplicação

O sistema é distribuído como aplicação containerizada, com seis serviços orquestrados por Docker Compose: a aplicação principal (FastAPI), o servidor de modelos locais Ollama, o módulo de avaliação, um banco de dados PostgreSQL, um serviço de fila de execuções e a interface *web*. Todos os experimentos foram disparados da mesma estação de trabalho, com processador AMD Ryzen 9 7950X3D e GPU NVIDIA GeForce RTX 4070 Ti Super de 16 GB, que condiciona o porte do maior modelo local que coube em memória.

A interface *web*, construída em Streamlit, expõe o sistema a usuários sem familiaridade com linha de comando. A Figura 3 mostra a tela para disparo de uma nova execução, na qual o usuário informa os metadados da coleção, escolhe o modo de *pipeline* e a granularidade de leitura, define os formatos de saída e fornece as imagens, por envio pelo navegador ou por diretório montado. As demais abas servem para acompanhar o andamento e o registro de cada execução, rodar o protocolo de avaliação e consultar o índice de registros extraídos. A consulta ao índice é feita por busca textual por campo. Com ela, a saída do *pipeline* se torna um índice consultável, e não apenas um arquivo exportado. Um mecanismo simples de autenticação protege a interface.

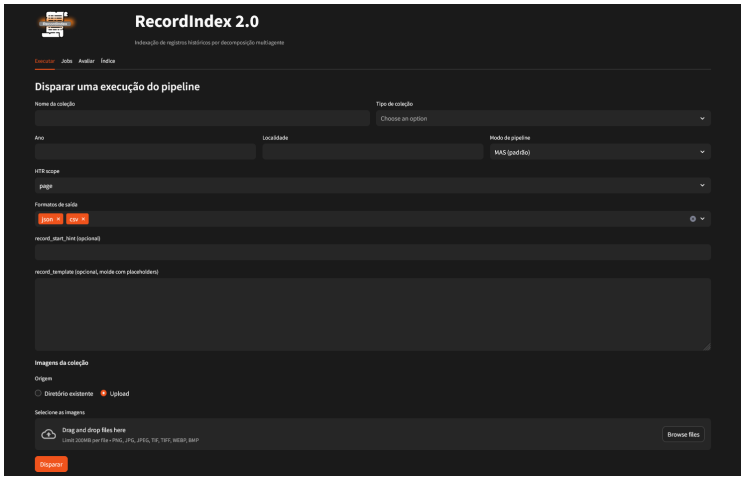


Figura 3 – Interface *web*, tela de disparo de execução.

Os resultados são exportados a partir do banco em JSON, CSV e TXT; o JSON preserva a hierarquia documental, o que permite rastrear cada entidade até sua imagem de origem.

As três configurações comparadas neste trabalho são também opções de uso, assim como a escolha entre executar tudo localmente ou recorrer a um provedor comercial. Os *prompts* e os demais parâmetros ficam em arquivo e em variáveis de ambiente, o que permite repetir experimentos sem editar código. O código e as configurações utilizadas nos experimentos estão disponíveis no repositório do projeto.[1]

4 Resultados e discussão

4.1 Configuração dos Experimentos

Os experimentos foram executados sobre a coleção de batismos de Porto da Cruz descrita na Introdução, com 132 registros no *ground truth*. Foram realizadas nove execuções sobre a coleção completa, cruzando os três modelos com os três modos de *pipeline*. O *Claude Sonnet 4.6* (`claude-sonnet-4-6`) é acessado por API; o *qwen3.5:397b*, modelo aberto, pelo Ollama em nuvem; e o *qwen3.5:9b* é executado localmente via Ollama. Em cada execução, o mesmo modelo opera em todos os agentes que dependem de LLM, de modo que, entre as execuções, variem o modelo e a configuração de *pipeline*, enquanto o *ground truth*, o módulo de avaliação e os limiares de casamento permanecem idênticos.

4.2 Resultados

A Tabela 4 apresenta as nove execuções sobre a coleção completa.

Com o *Claude Sonnet 4.6*, o modo generalista supera as duas configurações multiagente nas duas camadas de avaliação. Ele obteve F1 de segmentação de 0,913 contra 0,835 da leitura por linha e 0,729 da leitura por página, e média de extração de 0,823 contra 0,732 e 0,797. O generalista lidera em pai e data, e a leitura por página em nome e mãe. Com esse modelo, portanto, a arquitetura multiagente não superou o generalista nessa execução. A diferença concentra-se na delimitação de registros, e o número de registros

[1] https://github.com/agleizer/RecordIndex2

Métrica	***qwen3.5:9b***			***qwen3.5:397b***			***claude-sonnet-4.6***		
	linha	página	gen.	linha	página	gen.	linha	página	gen.
F1 de segmentação	0,392	0,635	**0,695**	0,592	**0,825**	0,824	0,835	0,729	**0,913**
Precision de segmentação	0,532	0,569	**0,709**	0,608	0,755	**0,854**	0,845	0,592	**0,910**
Recall de segm. (cobertura)	0,311	**0,720**	0,682	0,576	**0,909**	0,795	0,826	**0,947**	0,917
Registros produzidos (GT = 132)	77	167	127	125	159	123	129	211	133
Razão de segmentação	0,583	1,265	0,962	0,947	1,205	0,932	0,977	1,599	1,008
$F1_{\text{ef}}$ nome	0,407	**0,752**	0,714	0,574	**0,854**	0,831	0,805	**0,841**	0,833
$F1_{\text{ef}}$ pai	0,200	0,466	**0,514**	0,510	**0,758**	0,684	0,761	0,855	**0,878**
$F1_{\text{ef}}$ mãe	0,094	**0,429**	0,364	0,398	**0,708**	0,678	0,600	**0,793**	0,757
$F1_{\text{ef}}$ data	0,046	0,398	**0,434**	0,558	0,675	**0,734**	0,763	0,699	**0,822**
Média de extração	0,187	**0,511**	0,507	0,510	**0,749**	0,732	0,732	0,797	**0,823**
R_{ef} nome	26,5%	**64,4%**	57,6%	43,9%	**79,5%**	74,2%	73,5%	78,0%	**79,5%**

Tabela 4 – Nove execuções sobre a coleção completa de 132 registros.

produzidos torna isso evidente. O modo generalista produziu 133 registros para os 132 do *ground truth*. A leitura por página produziu 211, o que lhe dá a melhor cobertura do conjunto, 0,947, ao custo de uma *precision* de segmentação de 0,592; a leitura por linha produziu 129, com fronteiras deslocadas, de modo que apenas 109 encontraram par.

Com o modelo local de pequeno porte, a variável que decide é a granularidade da leitura. A leitura por linha fica em 0,392 de F1 de segmentação e 0,187 de média de extração, com cobertura de 0,311, ou seja, 41 dos 132 registros encontrados, enquanto a leitura por página e o modo generalista se mantêm em patamar semelhante entre si, 0,635 e 0,695 de F1 de segmentação, e 0,511 e 0,507 de média de extração. A leitura por linha é o modo mais decomposto de todos, com uma chamada de HTR por linha, e é o pior. Um recorte de linha oferece pouco contexto visual, e o exame dessas transcrições mostra o efeito: aparecem nomes e datas sem correspondência no manuscrito nem no *ground truth*. O mesmo modelo, recebendo a página inteira, produz transcrição mais coerente.

Com o *qwen3.5:397b*, a leitura por página e o modo generalista empatam na delimitação de registros, com F1 de segmentação de 0,825 e 0,824, mas a leitura por página fica à frente na extração, com média de 0,749 contra 0,732. O generalista identifica 123 registros para os 132 do *ground truth*, contra 159 da leitura por página, e é o de maior *precision* de segmentação entre os três modos desse modelo, 0,854.

Mantidos os sete agentes e variada apenas a granularidade, trocar a leitura por linha pela leitura por página eleva a média de extração em 0,324 com o *qwen3.5:9b*, em 0,239 com o *qwen3.5:397b* e em 0,065 com o *Claude Sonnet 4.6*. A taxa de recuperação efetiva do nome acompanha, com ganhos de 37,9, 35,6 e 4,5 pontos percentuais. O recorte de linha retira do modelo o contexto ao redor de cada trecho, e os resultados com os três modelos indicam que os modelos de melhor desempenho compensam parte dessa perda: nas configurações avaliadas, a diferença entre leitura por linha e por página diminuiu nos modelos de melhor desempenho. Com o *qwen3.5:9b*, essa é a diferença entre um resultado inaproveitável e um resultado utilizável.

A Figura 4 reúne as nove execuções nas duas camadas de avaliação. A aproximação das três séries da esquerda para a direita no painel de extração é representação do efeito descrito acima. O painel de segmentação indica que a leitura por página é a única série que não acompanha essa progressão. O eixo horizontal ordena os três modelos por desempenho observado, e as inclinações não devem ser lidas como função do número de parâmetros.

A Figura 5 ilustra a passagem de um registro pelos estágios do *pipeline*. O registro

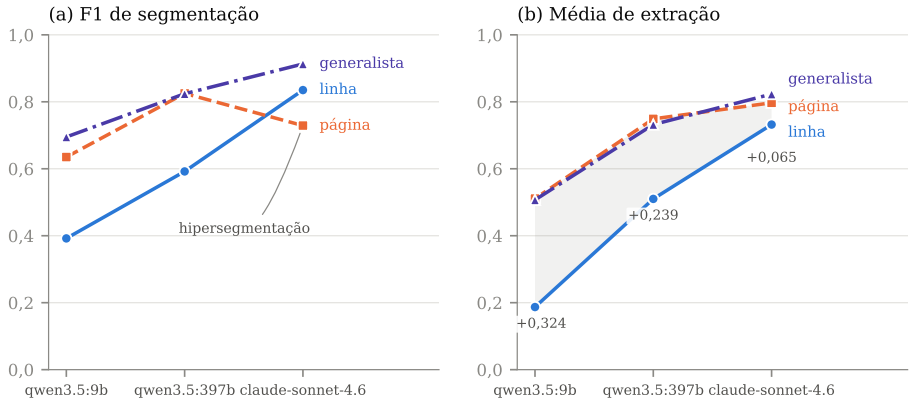
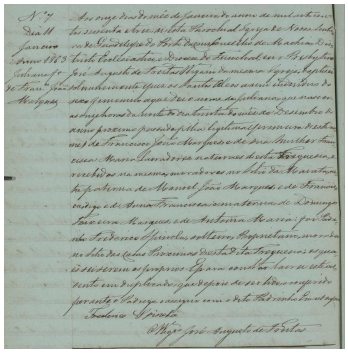


Figura 4 – Desempenho das nove execuções nas duas camadas de avaliação. A área sombreada em (b) marca o ganho de trocar a leitura por linha pela leitura por página.

escolhido corresponde ao assento nº 7 da coleção e obteve casamento completo em nome, pai e mãe (*Jaro-Winkler* igual a 1,000 nos três campos) contra o *ground truth* arquivístico. O painel (a) apresenta a imagem original; (b) o registro tal como o modo generalista o delimitou e transcreveu; e (c) os campos extraídos, validados com pontuação máxima.



(a) Assento nº 7.

Aos onze dias do mês de Janeiro, do anno de mil oitocentos sessenta e tres, n'esta Parochial Igreja de Nossa Senhora de Guadelupe do Porto da Cruz [...] a que dei o nome de **Juliana** [...] filha legítima (primeira deste nome) de **Francisco João Marques**, e de sua Mulher **Francisca Maria**, Lavradores naturaes desta Freguesia [...]

(b) Registro segmentado, com ortografia de época preservada.

```
{"nome": "Juliana",
 "pai": "Francisco João Marques",
 "mae": "Francisca Maria",
 "data": "11 de Janeiro de 1863"}
```

(c) Campos extraídos.

Figura 5 – Registro percorrendo os estágios do *RecordIndex*.

4.3 Origem das Falhas

A avaliação em duas camadas permite caracterizar a origem das falhas. A Tabela 5 detalha essa origem para o campo nome nas nove execuções, cuja falha é a mais impactante. De um lado ficam os registros que a segmentação não casou; de outro, os que foram delimitados corretamente, mas tiveram o nome extraído errado ou deixado em branco.

A origem dominante do erro depende da configuração. Na leitura por linha, a maior parte das falhas ainda vem da segmentação de registros, e o padrão se acentua nos modelos de menor desempenho: 66% com *Claude Sonnet*, 76% com o *qwen3.5:397b* e 94% com o modelo local. Nas melhores configurações o quadro se inverte, e o que sobra é erro de extração, sinal de que a delimitação deixou de ser o gargalo. No modo generalista com o *qwen3.5:397b*, 79% das falhas vêm da segmentação, não por dificuldade de extração, mas porque a configuração subsegmentou e deixou 27 registros sem par. Sobre os registros que chegou a delimitar, ela errou o nome em apenas 6,7% dos casos.

A vantagem do generalista na segmentação tem explicação plausível. Ele decide as

Modelo	Configuração	Falhas de nome	Por segmentação	Por extração
qwen3.5:9b	Leitura por linha	97	91 (94%)	6 (6%)
	Leitura por página	47	37 (79%)	10 (21%)
	Modo generalista	56	42 (75%)	14 (25%)
qwen3.5:397b	Leitura por linha	74	56 (76%)	18 (24%)
	Leitura por página	27	12 (44%)	15 (56%)
	Modo generalista	34	27 (79%)	7 (21%)
claude-sonnet-4.6	Leitura por linha	35	23 (66%)	12 (34%)
	Leitura por página	29	7 (24%)	22 (76%)
	Modo generalista	27	11 (41%)	16 (59%)

Tabela 5 – Origem das falhas no campo nome, nas nove execuções.

fronteiras vendo a imagem da página, ao passo que o A3 as decide a partir do texto já transcrito. E a transcrição do A2 é uma sequência de caracteres, sem posição na página: nela desapareceram os sinais que um leitor humano usa para saber onde um assento termina e outro começa. O recuo da primeira linha, o espaço maior entre um registro e o seguinte, o número do assento na margem e as notas laterais estão na imagem, e apenas nela. A hipótese é consistente com o padrão de erro: sem essas pistas, o A3 fragmenta em excesso quando recebe a página inteira, produzindo 211 registros para 132, e funde registros vizinhos quando recebe linhas isoladas.

4.4 Comparação com o *RecordIndex* 1.0

A Tabela 6 compara a melhor configuração do *RecordIndex* 2.0, executada com *Claude Sonnet 4.6* no modo generalista, com a versão anterior do sistema, sobre a mesma coleção de 132 registros.

Métrica	1.0 (sem LLM)	1.0 (com LLM)	2.0
Nomes corretos/idênticos	53/132 (40,2%)	47/132 (35,6%)	64/132 (48,5%)
Nomes reconhecíveis (*fuzzy*)	15/132 (11,4%)	12/132 (9,1%)	41/132 (31,1%)
Total “utilizável”	68/132 (51,5%)	59/132 (44,7%)	**105/132 (79,5%)**
Campos extraídos	apenas nome	apenas nome	nome, pai, mãe, data
Avaliação	manual	manual	automática

Tabela 6 – Comparação entre *RecordIndex* 1.0 e 2.0 (mesma coleção, 132 registros)

Essa comparação é indicativa, não exata. O *RecordIndex* 1.0 é outro software, avaliado em outro estudo (Gleizer; Oliveira, 2025) e sob critério distinto, com os nomes classificados manualmente como “correto”, “reconhecível” ou “erro”, categorias em que “reconhecível” equivale aproximadamente ao *fuzzy match* do *RecordIndex* 2.0 (JW $\geq$ 0,85). Os dois sistemas diferem em reconhecedor, estratégia de segmentação e critério de avaliação, de modo que a comparação mede o efeito conjunto dessas mudanças.

Ainda assim, a direção do resultado é clara: o *RecordIndex* 2.0 identificou 79,5% dos nomes sob critério *fuzzy*, contra 51,5% no *RecordIndex* 1.0 sem LLM, e superou a versão anterior mesmo sob correspondência idêntica, 48,5% contra 40,2%.

4.5 Discussão dos Achados

O ganho sobre a versão anterior não vem apenas da troca de CNN por LLM multimodal: depende também do desempenho do modelo empregado e da granularidade da leitura.

A degradação de nomes próprios observada no *RecordIndex* 1.0 não se repetiu. Foi para evitá-la que a correção estrutural (A4) foi separada da extração de entidades (A5), e o modo generalista, mesmo sem etapa de correção, obteve a melhor taxa de recuperação efetiva do nome, o que indica que parte do problema estava na correção. No *RecordIndex* 2.0, os piores resultados para nomes ocorreram com o modelo local lendo linha a linha, em que a própria transcrição já traz nomes sem correspondência no manuscrito.

Quanto ao grau de decomposição, não houve ganho consistente de desempenho nas configurações avaliadas: a arquitetura de sete agentes não superou o modo generalista com nenhum dos três modelos. Parte da margem do generalista sobre a leitura por página, porém, vem da hipersegmentação daquela execução, e uma única execução por configuração não permite dizer que a decomposição seja prejudicial, apenas que não foi vantajosa.

Com o modelo local, a granularidade de leitura influenciou mais o desempenho do que o arranjo dos agentes. Trocar a leitura por linha pela leitura por página, mantidos os mesmos sete agentes e o mesmo modelo, aumenta o F1 de segmentação em 0,243. Trocar o arranjo multiagente pelo generalista, mantida a granularidade, move 0,060. A primeira comparação isola uma variável; a segunda contrasta dois arranjos que diferem em mais de um aspecto, conforme a Seção 3.2. Ainda assim, a diferença associada à mudança de granularidade foi maior do que a diferença observada entre os dois arranjos arquiteturais.

Nas configurações avaliadas, a decomposição não apresentou ganho consistente de desempenho, mas tem duas propriedades que podem pesar na escolha. A primeira é permitir modelos diferentes por etapa, por exemplo um modelo mais capaz apenas na transcrição; neste trabalho todas as etapas usaram o mesmo modelo, e essa possibilidade não foi avaliada. A segunda é o tratamento de registros que atravessam a fronteira entre páginas, com o A0 acumulando o texto e extraindo os campos uma vez só, no fechamento, enquanto o generalista extrai duas vezes e reconcilia.

4.6 Limitações e Trabalho Futuro

Foi avaliada uma única coleção, de um ano e de um só tipo de registro. A parametrização por tipo de coleção descrita na Seção 3.2 existe em código e foi testada em amostras pequenas, mas não fez parte dos experimentos principais. Executar o sistema sobre outras coleções mostraria se as diferenças observadas se mantêm. Vale registrar ainda que a comparação entre o *RecordIndex* 2.0 e o 1.0 é indireta, conforme a Seção 3.1.

As execuções foram realizadas uma vez por configuração, sem repetição, o que impede estimar a variabilidade entre execuções de um mesmo arranjo. Como o mecanismo de decisão dos agentes é um modelo de linguagem, cuja saída não é determinística, parte das diferenças observadas entre configurações pode decorrer dessa variabilidade. Repetir cada configuração e reportar a dispersão permitiria separar diferença de arranjo de ruído de execução.

Nenhum modelo menor que o *qwen3.5:9b* foi testado em escala, e os três avaliados não formam uma escala controlada: diferem em porte, em família e em forma de acesso, e o número de parâmetros do modelo proprietário não é público, de modo que o comportamento muda entre eles sem que se possa atribuir a mudança a uma quantidade específica de parâmetros. A faixa dos modelos menores é a que mais muda, com arquiteturas de mistura de especialistas (MoE, do inglês *Mixture of Experts*) elevando o desempenho desse porte a cada lançamento, e testá-los verificaria uma hipótese de consequência prática: se não processarem a página inteira, a leitura por linha volta a ser a única alternativa. Na direção

oposta, o *qwen3.5:397b* foi acessado em nuvem por não caber na memória da GPU utilizada; executá-lo em máquina capaz de hospedá-lo mostraria se o desempenho se mantém e a que custo.

Por fim, a comparação entre a arquitetura multiagente e o modo generalista contrasta dois arranjos que diferem em mais de um aspecto, e não o número de agentes isoladamente, como discutido na Seção 3.2. Variar sistematicamente o grau de decomposição é a continuação mais direta desta investigação: sete agentes podem ser mais do que a tarefa exige e três podem ser menos, e uma configuração intermediária, com agentes distintos apenas para transcrição e extração, não foi avaliada.

Os resultados deste trabalho foram submetidos à Revista de Informática Teórica e Aplicada (RITA), do Instituto de Informática da Universidade Federal do Rio Grande do Sul, e aguardam avaliação.

5 Conclusão

Este trabalho construiu e avaliou três estratégias para a transcrição e indexação de registros genealógicos manuscritos, comparadas sobre a mesma coleção, com o mesmo *ground truth* e o mesmo protocolo de avaliação, tendo a versão anterior do sistema, baseada em CNNs, como referência.

O resultado central é que, nas configurações avaliadas, a decomposição em agentes não apresentou ganho consistente de desempenho, enquanto a granularidade de leitura se mostrou decisiva com o modelo local de menor porte, para o qual ler a página inteira em vez de linha a linha é a diferença entre um resultado inaproveitável e um resultado utilizável. Na melhor configuração o sistema obteve nomes utilizáveis em 79,5% dos registros, contra 51,5% na versão anterior, ampliando a extração de um para quatro campos e dispensando o treinamento supervisionado de que aquela versão dependia, o que importa em domínios com escassez de dados anotados. A principal ressalva é que essa comparação entre versões é indicativa, e que uma única execução por configuração não permite estimar a variabilidade das diferenças observadas.

Além do resultado empírico, o trabalho contribui com um protocolo de avaliação que separa as falhas de segmentação das de extração e torna possível comparar configurações completas de transcrição e indexação sobre o mesmo conjunto de referência.

Como artefato, o *RecordIndex* 2.0 é uma aplicação containerizada operacional, com persistência em banco de dados, fila de execuções e interface *web*, na qual as estratégias comparadas neste trabalho estão disponíveis como opção de configuração, assim como a escolha do provedor de modelo. Isso atende ao propósito que motivou o projeto desde a versão inicial, o de tornar essas técnicas utilizáveis por pesquisadores sem formação técnica em computação, e converte o resultado da comparação em decisão que o próprio usuário pode tomar conforme a infraestrutura de que dispõe.

Referências

ABADIE, N. *et al.* A Benchmark of Named Entity Recognition Approaches in Historical Documents Application to 19th Century French Directories. *In:* UCHIDA, S. *et al.* (ed.). **Document Analysis Systems**. Cham: Springer, 2022. p. 445–460. DOI: 10.1007/978-3-031-06555-2_30.

ALKENDI, W. *et al.* Advancements and Challenges in Handwritten Text Recognition: A Comprehensive Survey. **Journal of Imaging**, v. 10, n. 1, p. 18, jan. 2024. DOI: 10.3390/jimaging10010018. Acesso em: 18 set. 2025.

BERGMAN, M. K. The Deep Web: Surfacing Hidden Value. **Journal of Electronic Publishing**, Michigan Publishing, v. 7, n. 1, ago. 2001. DOI: 10.3998/3336451.0007.104. Acesso em: 5 set. 2025.

BIBLIOTECA PÚBLICA E ARQUIVO REGIONAL DA MADEIRA. **Livro de registro de baptismos do Porto da Cruz do ano de 1863 (PT/ABM/PMCH04/001/00022)**. Funchal: [*s. n.*], 1863. Disponível em: https://arquivo-abm.madeira.gov.pt/descriptions/4741.

BOILLET, M. *et al.* Multiple Document Datasets Pre-training Improves Text Line Detection With Deep Neural Networks. *In:* 2020 25th International Conference on Pattern Recognition (ICPR). [*S. l.*]: IEEE, jan. 2021. p. 2134–2141. DOI: 10.1109/ICPR48806.2021.9412447. Acesso em: 5 set. 2025.

BOROS, E. *et al.* Post-Correction of Historical Text Transcripts with Large Language Models. *In:* PROCEEDINGS of LaTeCH-CLfL 2024. St. Julians: Association for Computational Linguistics, mar. 2024. p. 133–159. DOI: 10.18653/v1/2024.latechclfl-1.14. Acesso em: 5 set. 2025.

COHEN, W. W. *et al.* A Comparison of String Distance Metrics for Name-Matching Tasks. *In:* PROCEEDINGS of the 2003 International Conference on Information Integration on the Web. [*S. l.*]: AAAI Press, 2003. p. 73–78. Disponível em: https://dl.acm.org/doi/proceedings/10.5555/3104278.

CROSILLA, G. *et al.* Benchmarking Large Language Models for Handwritten Text Recognition. **Journal of Documentation**, p. 1–21, ago. 2025. DOI: 10.1108/JD-03-2025-0082. Acesso em: 5 set. 2025.

DORRI, A. *et al.* Multi-Agent Systems: A Survey. **IEEE Access**, v. 6, p. 28573–28593, 2018. DOI: 10.1109/ACCESS.2018.2831228. Acesso em: 14 abr. 2026.

EHRMANN, M. *et al.* Named Entity Recognition and Classification in Historical Documents: A Survey. **ACM Comput. Surv.**, v. 56, n. 2, 27:1–27:47, set. 2023. DOI: 10.1145/3604931. Acesso em: 17 set. 2025.

GARRIDO-MUNOZ, C. *et al.* Handwritten Text Recognition: A Survey. **IEEE Transactions on Pattern Analysis and Machine Intelligence**, arXiv, v. 48, n. 4, p. 4367–4387, fev. 2025. DOI: 10.1109/TPAMI.2025.3646002. Acesso em: 5 set. 2025.

GLEIZER, A. M.; OLIVEIRA, I. C. de. RecordIndex: Um Sistema De Transcrição Automatizada De Registros Genealógicos Manuscritos. *In:* ISLA 2025 PROCEEDINGS. [*S. l.: s. n.*], ago. 2025. Disponível em: https://aisel.aisnet.org/isla2025/25. Acesso em: 16 set. 2025.

HILTMANN, T. *et al.* **NER4all or Context Is All You Need: Using LLMs for Low-Effort, High-Performance NER on Historical Texts. A Humanities Informed Approach**. [*S. l.*]: arXiv, fev. 2025. DOI: 10.48550/arXiv.2502.04351. Acesso em: 5 set. 2025.

HUMPHRIES, M. *et al.* Unlocking the Archives: Using Large Language Models to Transcribe Handwritten Historical Documents. **Historical Methods**, Routledge, v. 58, n. 3, p. 175–193, jul. 2025. DOI: 10.1080/01615440.2025.2500309. Acesso em: 17 set. 2025.

INUNGANBI, S. A Systematic Review on Handwritten Document Analysis and Recognition. **Multimedia Tools and Applications**, v. 83, n. 2, p. 5387–5413, jan. 2024. DOI: 10.1007/s11042-023-15326-9. Acesso em: 18 set. 2025.

ISOM, J. D. **An HTR-LLM Workflow for High-Accuracy Transcription and Analysis of Abbreviated Latin Court Hand**. [*S. l.*]: arXiv, jul. 2025. DOI: 10.48550/arXiv.2507.04132. Acesso em: 17 set. 2025.

KALRA, M. P. *et al.* LLM Powered HTR: Integrating Handwritten Text Recognition System with Large Language Model. *In:* 2024 IEEE Students Conference on Engineering and Systems. [*S. l.*]: IEEE, jun. 2024. p. 1–6. DOI: 10.1109/SCES61914.2024.10652342. Acesso em: 5 set. 2025.

KANERVA, J. *et al.* **OCR Error Post-Correction with LLMs in Historical Documents: No Free Lunches**. [*S. l.*]: arXiv, fev. 2025. DOI: 10.48550/arXiv.2502.01205. Acesso em: 5 set. 2025.

KIESSLING, B. *et al.* eScriptorium: An Open Source Platform for Historical Document Analysis. *In:* 2019 International Conference on Document Analysis and Recognition Workshops. [*S. l.*]: IEEE, set. 2019. v. 2, p. 19–19. DOI: 10.1109/ICDARW.2019.10032. Acesso em: 18 out. 2025.

KIM, S. *et al.* **Early Evidence of How LLMs Outperform Traditional Systems on OCR/HTR Tasks for Historical Records**. [*S. l.*]: arXiv, jan. 2025. DOI: 10.48550/arXiv.2501.11623. Acesso em: 5 set. 2025.

LEVCHENKO, M. **Evaluating LLMs for Historical Document OCR: A Methodological Framework for Digital Humanities**. [*S. l.*]: arXiv, out. 2025. DOI: 10.48550/arXiv.2510.06743. Acesso em: 16 nov. 2025.

MAKHOUL, J. *et al.* Performance Measures For Information Extraction. **Proceedings of DARPA Broadcast News Workshop**, ago. 2000.

MANNING, C. D. *et al.* Cap. 8, Evaluation in Information Retrieval. *In:* INTRODUCTION to Information Retrieval. [*S. l.*]: Cambridge University Press, jul. 2008. p. 151–176. DOI: 10.1017/CBO9780511809071. Acesso em: 16 set. 2025.

MUEHLBERGER, G. *et al.* Transforming Scholarship in the Archives through Handwritten Text Recognition: Transkribus as a Case Study. **Journal of**

Documentation, v. 75, n. 5, p. 954–976, jul. 2019. DOI: 10.1108/JD-07-2018-0114. Acesso em: 18 out. 2025.

NEUDECKER, C. *et al.* A Survey of OCR Evaluation Tools and Metrics. *In:* PROCEEDINGS of HIP '21. New York, ACM, out. 2021. p. 13–18. DOI: 10.1145/3476887.3476888. Acesso em: 17 set. 2025.

PUIGCERVER, J. Are Multidimensional Recurrent Layers Really Necessary for Handwritten Text Recognition? *In:* 14TH International Conference on Document Analysis and Recognition. [*S. l.*]: IEEE, nov. 2017. v. 01, p. 67–72. DOI: 10.1109/ICDAR.2017.20. Acesso em: 5 set. 2025.

RUSSELL, S.; NORVIG, P. **Artificial Intelligence: A Modern Approach**. 4. ed. Hoboken: Pearson, 2021.

TARRIDE, S.; MAARAND, M. *et al.* Large Scale Genealogical Information Extraction From Handwritten Quebec Parish Records. **IJDAR**, v. 26, n. 3, p. 255–272, set. 2023. DOI: 10.1007/s10032-023-00427-w. Acesso em: 5 set. 2025.

TARRIDE, S.; SCHNEIDER, Y. *et al.* **Improving Automatic Text Recognition with Language Models in the PyLaia Open-Source Library**. [*S. l.*]: arXiv, abr. 2024. DOI: 10.48550/arXiv.2404.18722. Acesso em: 18 out. 2025.

TEKLIA. **Arkindex versus the Other Platforms**. [*S. l.*]: TEKLIA, abr. 2024. https://teklia.com/our-solutions/arkindex-comparison/. Acesso em: 18 out. 2025.

TEKLIA. **Arkindex, the Document Processing Platform**. [*S. l.*]: TEKLIA, fev. 2023. https://teklia.com/our-solutions/arkindex/. Acesso em: 18 out. 2025.

XI, Z. *et al.* **The Rise and Potential of Large Language Model Based Agents: A Survey**. [*S. l.: s. n.*], set. 2023. DOI: 10.48550/arXiv.2309.07864. Acesso em: 14 abr. 2026.